

Memory Management

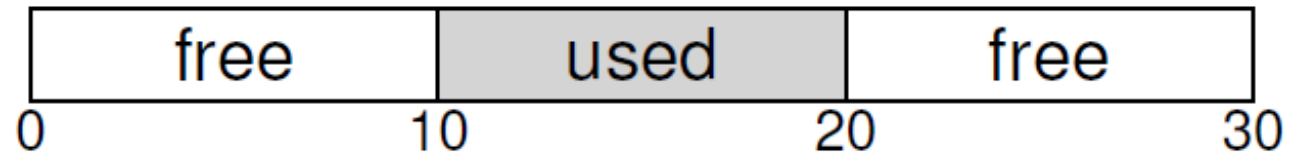
Free Space Management

CS232 Operating Systems, Fall 2019

Taj Muhammad Khan

External Fragmentation is a problem

- Allocated memory segments usually are of unequal size
- As memory portions are allocated and deallocated, the memory quickly becomes full of small “holes”
- Consider the example:



- What if a request for 15 bytes arrive?

```
void *malloc(size_t size)
```

```
void free(void *ptr)
```

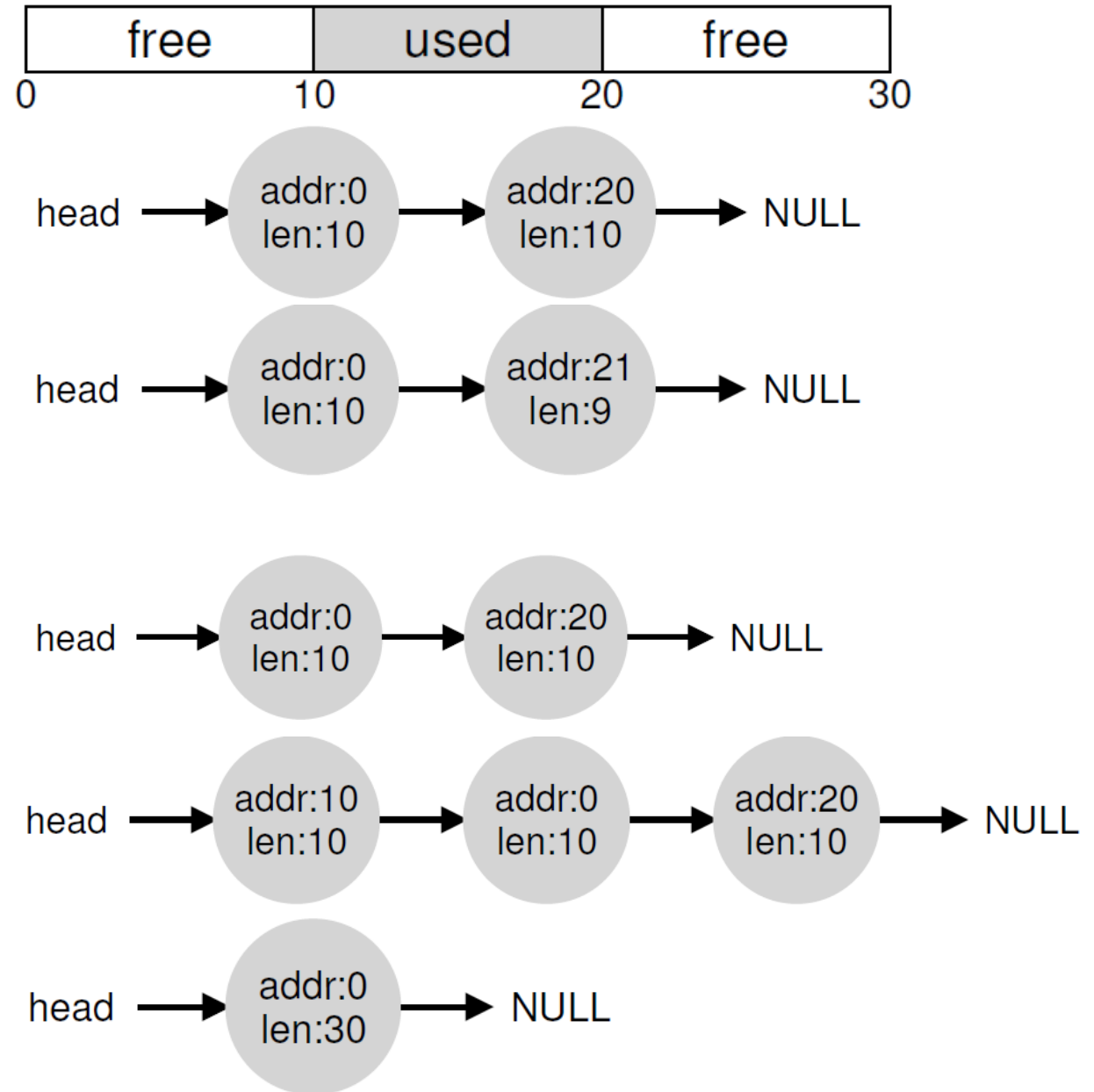
- Notice that free doesn't take a size parameter.
- How does it know how big is the chunk being free'd?
- The allocator library maintains sort of a *free list*

Assumptions

- The allocator manages a continuous region of bytes
- Compaction is not possible

Mechanisms

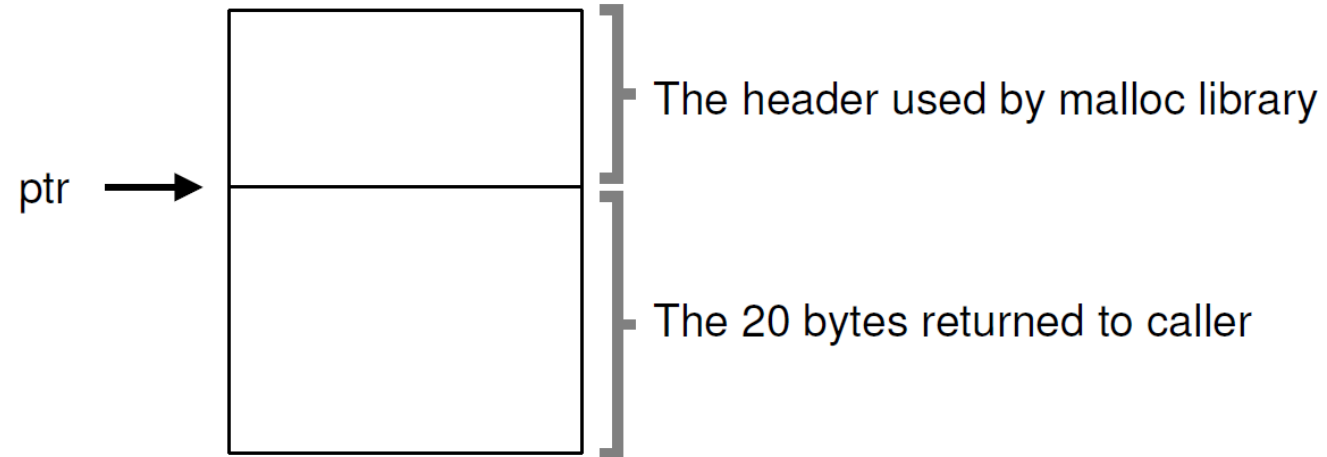
- Splitting
 - Assume a 30 bytes heap
 - Request for 10 bytes is easy
 - Request for <10 bytes?
 - Request for 1 byte
- Coalescing
 - Free (10)
 - Entire list is free but unavailable!
 - Coalescing



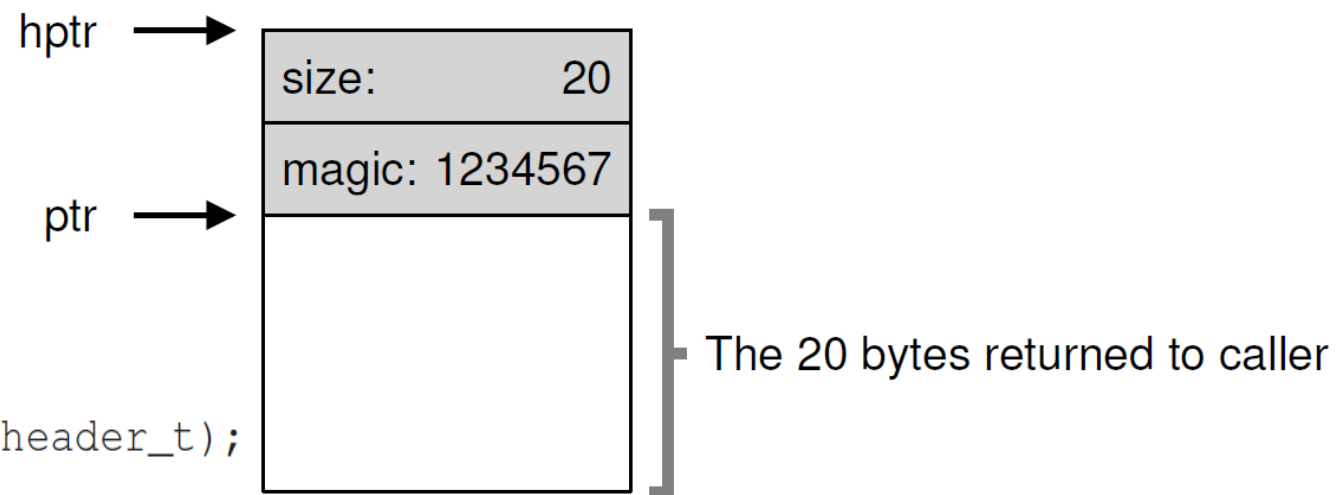
Book keeping

- Tracking size of allocated blocks

```
ptr = malloc(20);
```



```
typedef struct __header_t {  
    int size;  
    int magic;  
} header_t;
```



```
void free(void *ptr) {  
    header_t *hptr = (void *)ptr - sizeof(header_t);  
    ...  
}
```

Embedding a free list

- We've to keep track of allocations via malloc
 - Usually in a list when making a new node we call malloc()
 - Here we can't. Why?
-
- We need to build the free list inside the free space itself!

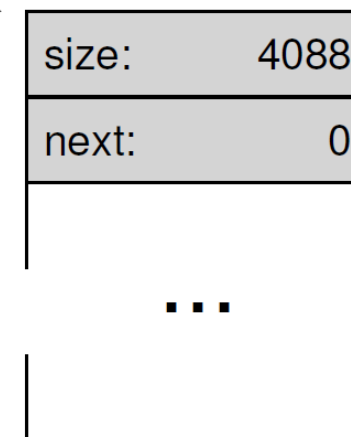
Embedding a free list ... contd.

- Assume we need to manage a 4096-byte memory
- Initially the list will have one entry:
 - One node of size 4096 – sizeof(head)

```
typedef struct __node_t {  
    int size;  
    struct __node_t *next;  
} node_t;
```

```
// mmap() returns a pointer to a chunk of free space  
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,  
                    MAP_ANON|MAP_PRIVATE, -1, 0);  
  
head->size = 4096 - sizeof(node_t);  
head->next = NULL;
```

head →



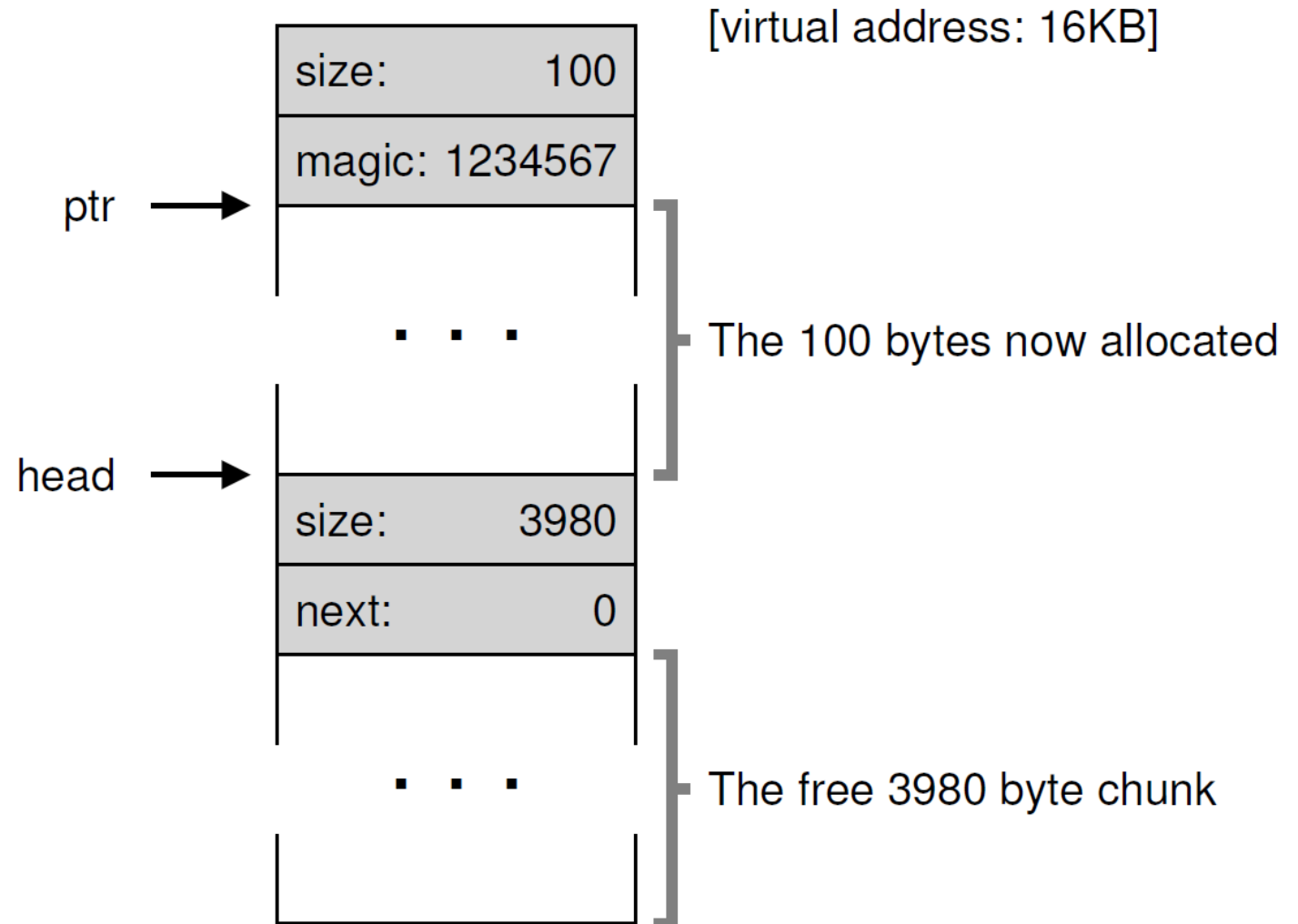
[virtual address: 16KB]
header: size field

header: next field (NULL is 0)

the rest of the 4KB chunk

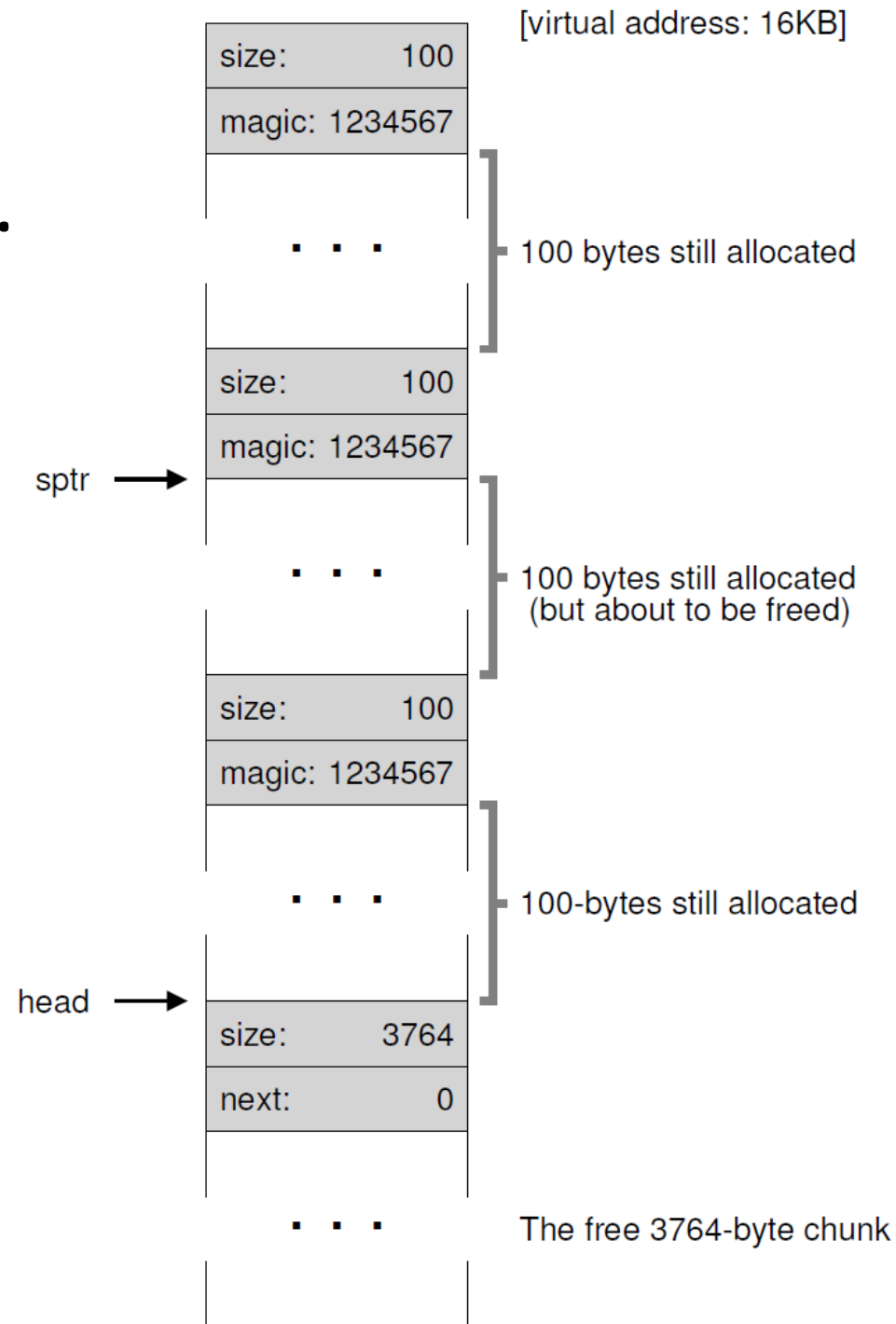
Embedding a free list ... contd.2

- After calling malloc(100)
 - A split was done.
- Upon request of 100 bytes, 108 were actually allocated



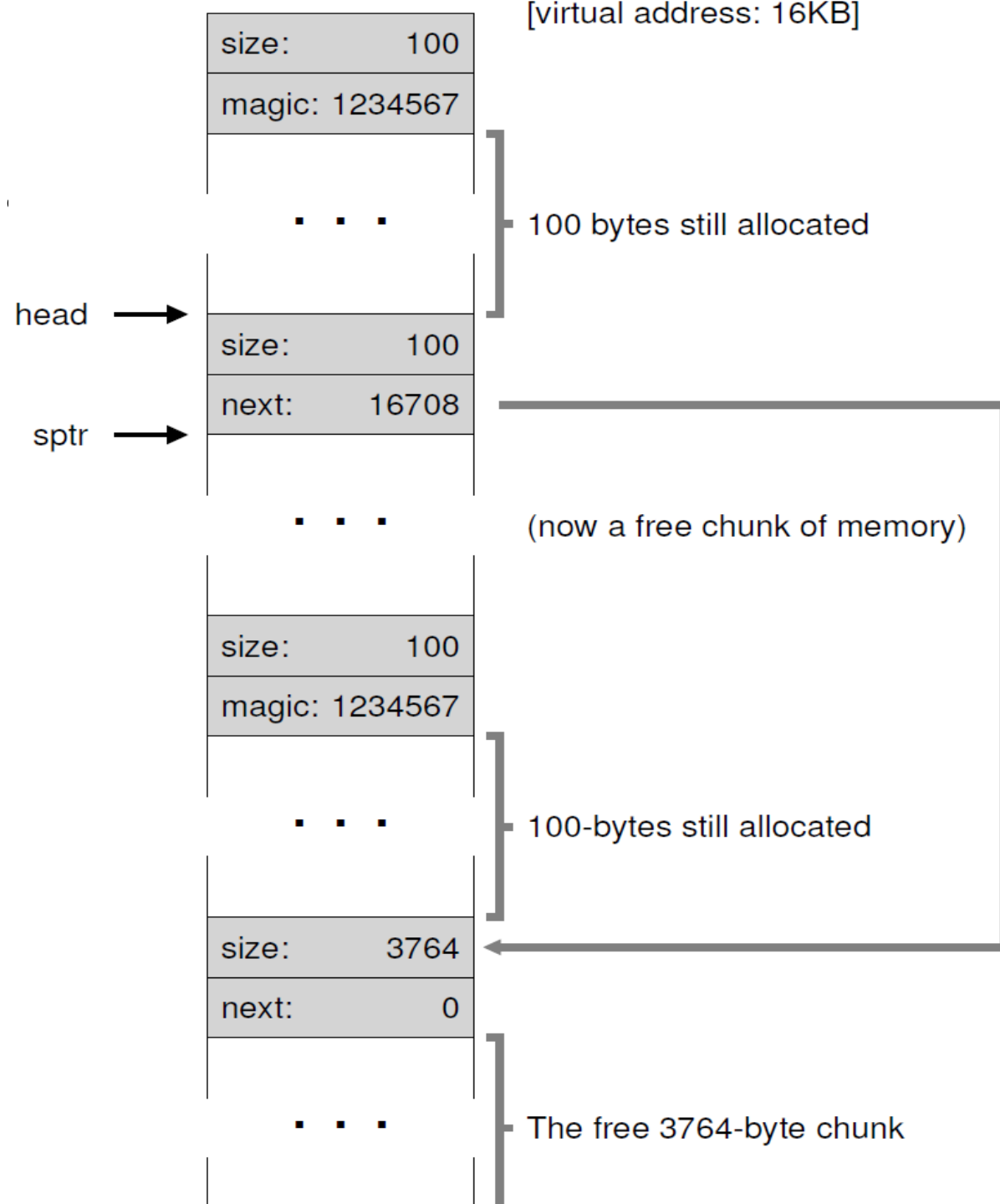
Embedding a free list ... contd.

- Three blocks of 100 bytes allocated
 - Figure 17.5



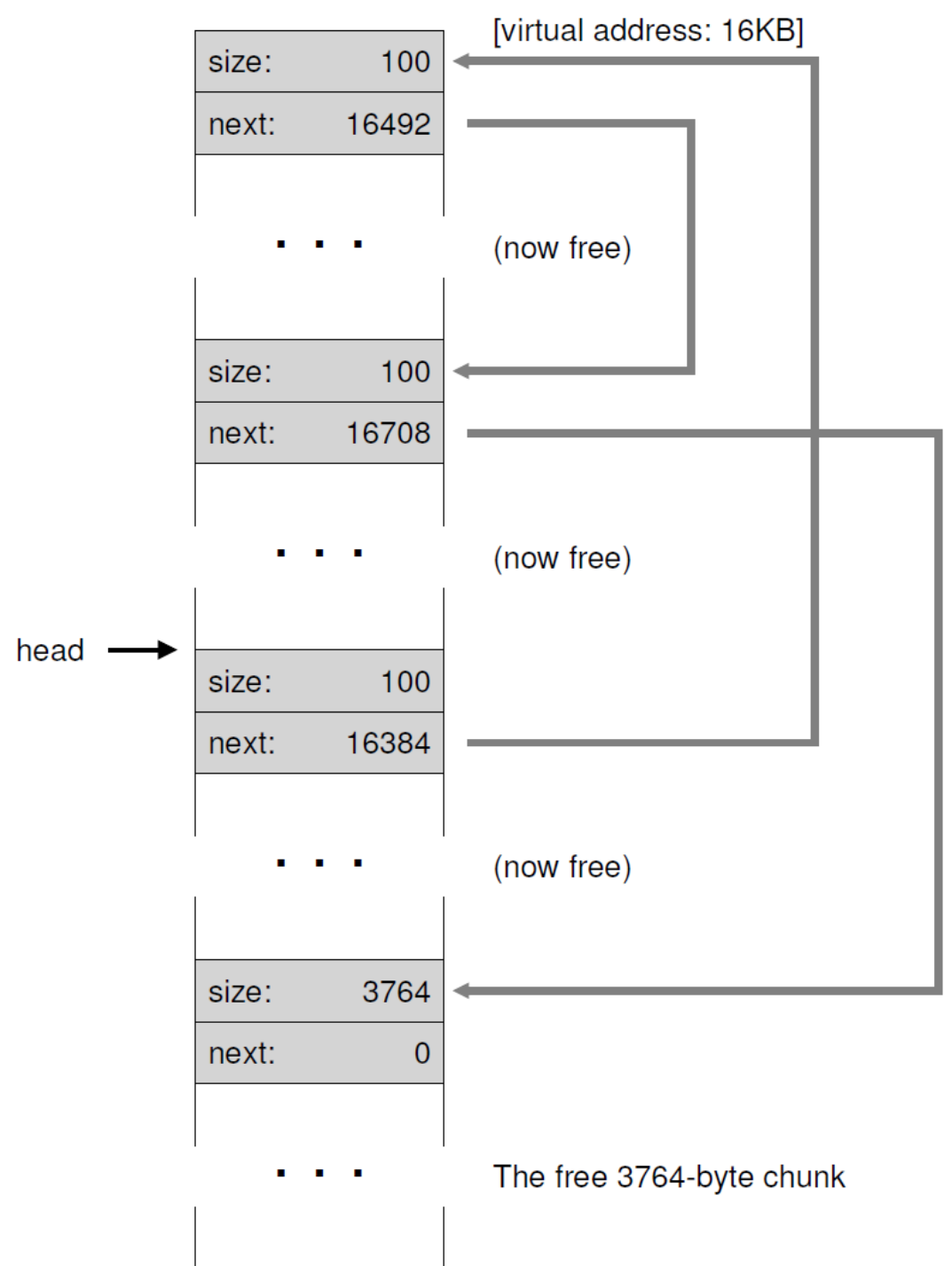
Embedding the free list .

- Free a chunk of 100 bytes
 - Figure 17.6
 - Free list has more than one element
 - Is fragmented



Embedding a free list ...

- All blocks free'd
 - Figure 17.7
 - List is all chopped up
 - Coalescing can be a solution

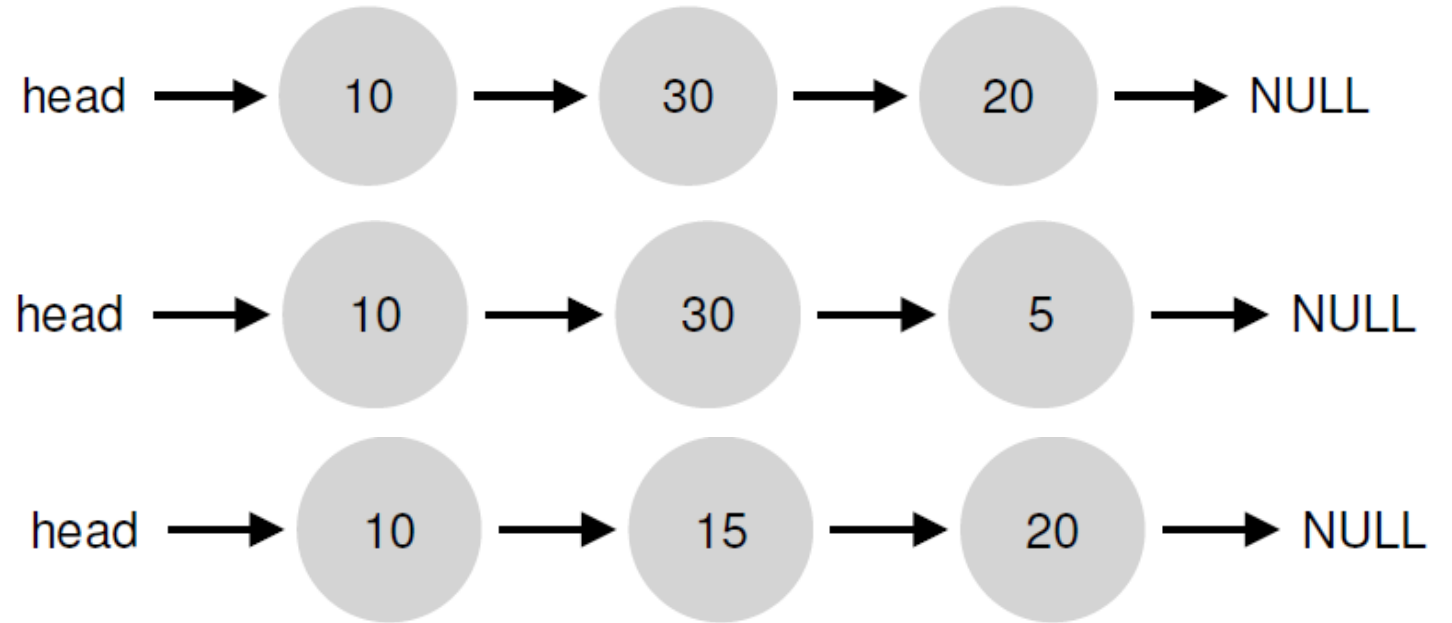


Block allocation strategies (policies)

- Best fit
 - Return the smallest block that satisfies our request
- Worst fit
 - Return the biggest chunk that satisfies our request
- First fit
 - Return the first chunk that satisfies our request
- Next fit
 - Return the next block (from the last block allocated) that satisfies our request

Examples

- Original list
- Best fit
- Worst fit
- First fit
- Next fit

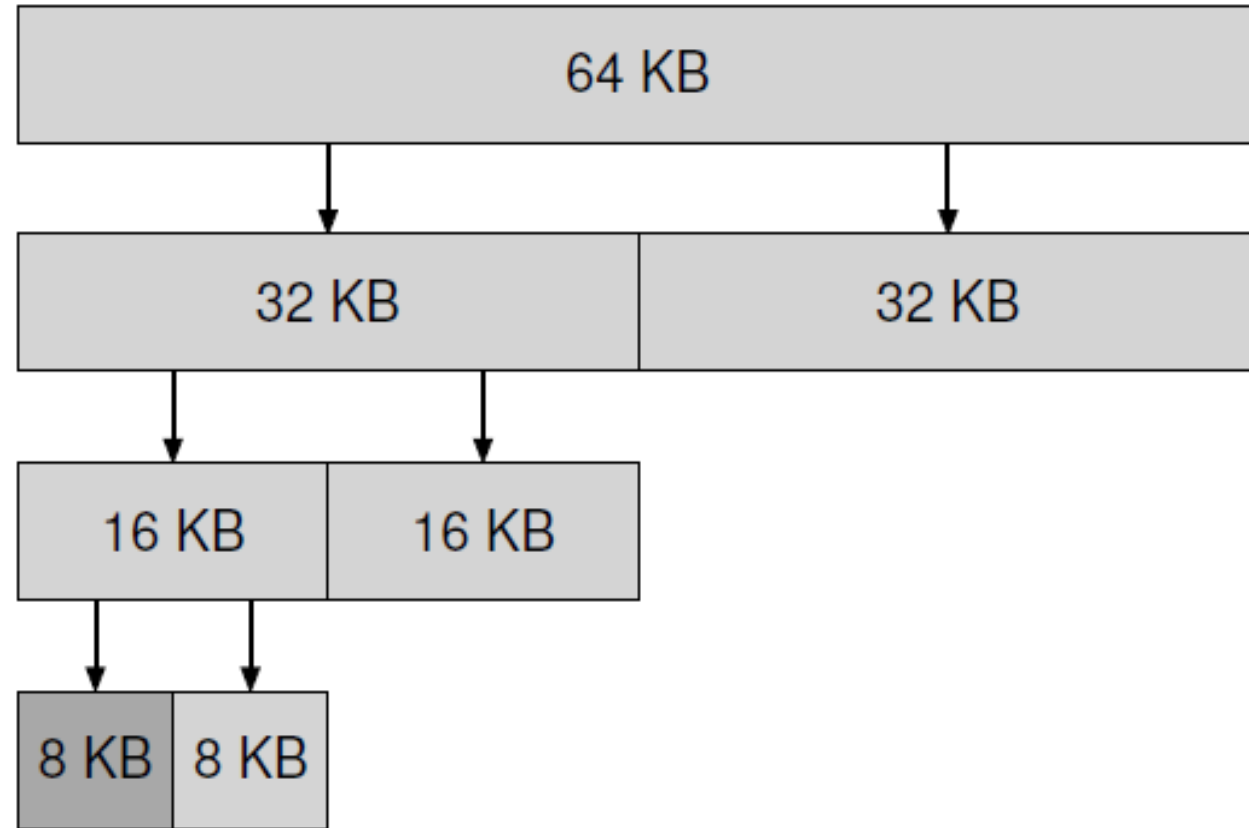


Other approaches

- Segregated lists
 - Keep a separate list of frequently-requested-objects-sized blocks
 - Less fragmentation
- Slab allocator in Solaris
 - Allocates object-caches for frequently requested kernel structures i.e locks, file system inodes, etc.
 - When object-caches run low, they request more *slabs* from *general allocator*
 - When they have too few allocated objects, the general allocator can reclaim memory
 - It keeps free objects in pre-initialized state

Other approaches

- Binary Buddy Allocator
- Memory is a space of 2_N
- When a request arrives
 - The space is recursively divided by 2
 - Until we reach a size that satisfies
 - Further division will be too small
 - Here a request of 7 bytes
 - May suffer from internal fragment.



Other approaches

- Binary Buddy Allocator ... contd.
- What happens when a block is free'd?
- We see if its buddy is free, coalesce them!
- Continue upper level, recursively!
- Free() only receives address; so how do we determine the address of buddy?

